

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)**

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №2

по курсу «Технологии параллельного программирования»

Обработка изображений на GPU. Фильтры.

Выполнил: Ахметшин Б.Р.

Группа: М8О-403Б-22

Преподаватели: А.Ю. Морозов,

Е.Е. Заяц

Москва, 2026

Условие

Цель работы: научиться использовать GPU для обработки изображений. Использование текстурной памяти и двухмерной сетки потоков.

Выполненная реализация: фильтр выделения границ Превитта для изображения в пользовательском бинарном формате (**w**, **h**, затем пиксели **rgba**).

Программное и аппаратное обеспечение

GPU

Имя	NVIDIA Tesla T4
Compute Capability	7.5
Объем глобальной памяти	15637086208
Разделяемая память на блок	49152
Константная память	65536
Регистров на блок	65536
Макс. потоков на блок	(1024, 1024, 64)
Макс. размер сетки	(2147483647, 65535, 65535)
Количество мультипроцессоров	40

CPU

Архитектура	x86_64
CPU op-mode(s)	32-bit, 64-bit
Address sizes	48 bits physical, 48 bits virtual
Порядок байт	Little Endian
CPU(s)	12
Имя модели	AMD Ryzen 5 7600 6-Core Processor
Ядер / потоков	6 / 12

RAM

64 GB

ОС

Ubuntu 24.04.4 LTS

Компилятор

nvcc, g++

Метод решения

Изображение читается в массив `uchar4` (rgba), после чего копируется в `cudaArray` и связывается с `cudaTextureObject`. Ядро запускается на двумерной сетке потоков (блок 16x16), каждый поток обрабатывает один пиксель.

Для вычисления интенсивности используется перевод в оттенок серого:

$$gray = 0.299r + 0.587g + 0.114b.$$

Далее применяются маски Превитта по осям x и y ; итоговая величина градиента:

$$g = \sqrt{g_x^2 + g_y^2},$$

после чего значение ограничивается диапазоном $[0, 255]$ и записывается в выходной пиксель как $r=g=b=g$, $a=0$.

Описание программы

Файлы реализации:

- `lab2.cu` — GPU-реализация на CUDA;
- `lab2-cpu.cpp` — CPU-реализация для сравнения;
- `converter.py` — конвертация между бинарным форматом ЛР и PNG.

В GPU-версии реализованы два вида измерений:

- `kernel_ms` — время ядра (через CUDA events);
- `total_ms` — совокупное время вычислительной части (от старта GPU-обработки до получения результата на CPU).

Оба значения логируются в файл `timings_gpu_lab2.txt` (или путь из переменной `LAB2_TIMING_FILE`).

CPU-версия логирует `total_ms` в файл `timings_cpu_lab2.txt` (или путь из `LAB2_TIMING_FILE`).

Сборка:

```
cd lab2
make
```

Запуск (GPU):

```
printf "in.data\nout_gpu.data\n" | ./lab2
```

Запуск (CPU):

```
printf "in.data\nout_cpu.data\n" | ./lab2-cpu
```

Результаты

Замеры:

No	image	CUDA (grid, block)	CPU, ms	GPU kernel, ms	GPU total, ms
1	512x512	<<<(64x64), (8x8)>>>	6.100761	0.134752	384.695764
2	512x512	<<<(32x32), (16x16)>>>		0.141344	382.040485
3	512x512	<<<(16x64), (32x8)>>>		0.138944	381.365321
4	4096x4096	<<<(512x512), (8x8)>>>	436.016163	1.592064	462.457037
5	4096x4096	<<<(256x256), (16x16)>>>		1.961760	502.916248
6	4096x4096	<<<(128x512), (32x8)>>>		1.838432	463.340353

Скриншоты (обязательное требование для ЛР2):



Рис. 1: Визуальное сравнение до и после обработки

Выводы

В лабораторной работе реализован фильтр обработки изображений на GPU с использованием текстурной памяти и двумерной сетки потоков, а также эквивалентная CPU-реализация для сравнения производительности.

По замерам видно, что время ядра GPU мало и хорошо масштабируется: для изображения 512×512 оно составляет 0.134–0.141 мс, а для 4096×4096 — 1.59–1.96 мс. Однако совокупное время GPU значительно выше из-за накладных расходов (копирование данных и служебные операции): 381–385 мс для 512×512 и 462–503 мс для 4096×4096 .

Сравнение с CPU показывает, что в текущей постановке CPU быстрее по общему времени: 6.10 мс против 381–385 мс для 512×512 , и 436.02 мс против 462–503 мс для 4096×4096 . При этом по времени самого ядра GPU значительно эффективнее, что подтверждает корректность распараллеливания, но также показывает доминирование накладных расходов в полном времени выполнения.

Лучшая конфигурация по общему времени: для 512×512 — $\lll 16 \times 64, 32 \times 8 \ggg$, для 4096×4096 — $\lll 512 \times 512, 8 \times 8 \ggg$.